

PROJECT SYNOPSIS

MovieMatch

A Hybrid Movie Recommendation System using Machine Learning (SVD, KNN) and Generative AI (Google Gemini)

Student Name(s)	Aarav Gupta	Roll Number(s)	2400320100006
Branch / Department	CSE	College Name	ABES ENGINEERING COLLEGE
Project Guide	Ms. Anshika Chaudhary	Academic Year	2026-27

1. Introduction

Streaming platforms now offer thousands of movies, and viewers often spend more time choosing what to watch than actually watching it. This problem, known as **choice paralysis**, is where recommender systems help: they predict what a user will enjoy from past ratings and from the behaviour of similar users.

MovieMatch is a Netflix-style movie discovery website that mixes machine learning with an AI chat assistant. Rating patterns are learned with **SVD** and **KNN**, while a Large Language Model (**Gemini 2.5 Flash** via OpenRouter) talks to the user and writes short synopses on request. The catalogue has grown from the 9,742 MovieLens titles to **80,000+ movies**, including Marvel, Bollywood and Pan-India hits, and the interface is a **React** single-page app.

2. Problem Statement

Choosing what to watch often takes longer than watching it. A plain recommendation list gives no reason for its picks, cannot be asked a question in everyday language, and does not help two people agree on one movie. Titles missing from the training data are left out altogether. A friendlier, conversational and shared-taste recommender is needed.

3. Objectives

- To compare **SVD**, item-based **KNN (Pearson baseline)** and a baseline model using RMSE, MAE and NDCG.
- To add a **Date Night** mode that finds movies two people will both enjoy.
- To add an **AI assistant**, on-demand synopses and a personalised welcome using an LLM.
- To grow the catalogue to **80,000+ movies** with cleaned titles and locally cached posters.
- To keep a layered **React / FastAPI / SQLite (3NF)** design with a validated REST API.
- To make choosing a movie quick through a cinematic, easy-to-browse interface.

4. Proposed Solution

MovieMatch is a three-tier web app. The browser shows a React interface, the FastAPI server does the work (database, trained models and AI calls), and the answer returns as JSON to be drawn as movie cards and carousels.

Feature	What it does
Top Picks	Suggests unseen movies the user is likely to rate highly, using SVD (or KNN / Baseline).
Date Night Matchmaker	Pick two users and get movies both should like; their scores are joined with a harmonic mean.
Franchise Rails	Spots sequels and prequels from a movie's title and shows them as "The Collection".
AI Assistant	A chat box that turns a plain request into a real movie card from the database.
AI Synopsis and Welcome	Short plot text when none exists, plus a greeting built from the user's favourite movies.
Profiles and My List	Username login, 1-5 star ratings and a Plan to Watch list.

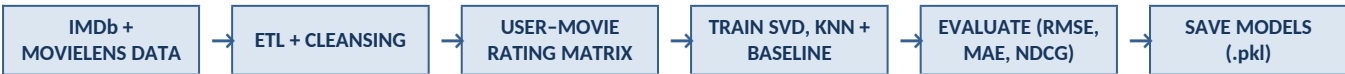
Target users: movie viewers, casual streamers and pairs choosing together. **Expected benefits:** quicker decisions, shared picks and conversational discovery.

5. Scope of the Project

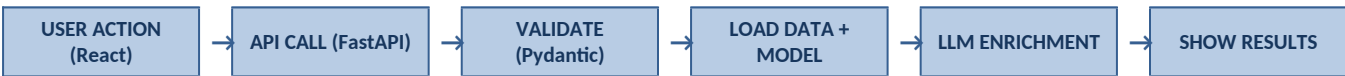
Included	Outside the current scope
SVD, KNN and Baseline recommendation models	Real-time / automatic model retraining
Date Night Matchmaker and franchise rails	Mobile application
LLM assistant, synopses and AI welcome	Video streaming or playback
80,000+ movie catalogue with poster caching	Social features and password-based login
REST API, 3NF database and React interface	Deep-learning models (planned as future work)

6. Methodology / Working

A. Offline: model training



B. Online: serving a request



SVD splits the rating matrix into hidden “taste” factors for users and movies; combining them predicts a rating for movies the user has not seen. **KNN** (item-based, Pearson baseline similarity) finds the movies most similar to those a user rated and recommends from that neighbourhood. On MovieLens, KNN scored best (RMSE 0.852, MAE 0.648), followed by Baseline (0.873, 0.671) and SVD (0.879, 0.673).

For **Date Night**, each user’s predicted score is combined as $2 \cdot s_1 \cdot s_2 / (s_1 + s_2)$. A pair of 5 and 1 gives only 1.67 while 3 and 3 gives 3.0, so movies loved by one person but disliked by the other drop down the list.

7. Technologies Used

Category	Technology
Programming Language	Python 3.10+ and JavaScript
Frontend	React, Vite, Tailwind CSS v4, React Router, Lucide-React
Backend	FastAPI, Uvicorn, Pydantic, Dependency Injection
Database	SQLite through SQLAlchemy 2.1 ORM
Machine Learning	scikit-surprise: SVD, KNNBaseline, BaselineOnly (saved as Pickle files)
Generative AI	Gemini 2.5 Flash via OpenRouter API with structured prompts
Data and Testing	MovieLens and IMDb data; OMDb API for posters; pytest and httpx

8. Modules

Module	Description
1. Frontend (UI)	React pages for Login, Home, Discover, My List, Top Picks and Date Night, with the floating AI assistant.
2. Backend API	FastAPI endpoints that validate input and coordinate the database, models, LLM and OMDb.
3. Data and ETL	3NF tables for users, movies, ratings and watchlist; scripts that import and clean the 80,000+ titles and cache posters.
4. Recommendation Engine	Pre-trained SVD, KNN and Baseline models, Date Night matching and franchise detection.
5. Generative AI Module	Assistant, synopsis generator and personalised welcome built on structured prompts.

9. Expected Outcome

The completed project will be a working full-stack application that:

- personalised Top Picks from trained SVD, KNN and Baseline models;
- Date Night picks for two people and automatic franchise collections;
- an AI assistant that answers in plain language, while the app keeps working if the LLM or poster service fails; and
- a clean, modular code base that is easy to extend and deploy.

10. Future Scope

- Automated nightly ML retraining pipeline so models stay up to date.
- Neural Collaborative Filtering (deep learning) for higher recommendation quality.
- Cloud deployment with a managed PostgreSQL database.
- Caching of LLM responses to reduce latency and API cost.
- Mobile application and secure password-based user accounts.
- AI Director: an LLM-powered virtual director that curates personalised picks in natural language.

11. References

- [1] Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," IEEE Computer, vol. 42, no. 8, 2009.
- [2] F. Ricci, L. Rokach and B. Shapira (Eds.), Recommender Systems Handbook, Springer.
- [3] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM Trans. Interact. Intell. Syst., vol. 5, no. 4, 2015.
- [4] N. Hug, "Surprise: A Python library for recommender systems," J. Open Source Softw., vol. 5, no. 52, 2020.
- [5] FastAPI Documentation – <https://fastapi.tiangolo.com>
- [6] React Documentation – <https://react.dev>
- [7] Tailwind CSS Documentation – <https://tailwindcss.com/docs>
- [8] SQLAlchemy Documentation – <https://docs.sqlalchemy.org>
- [9] Pydantic Documentation – <https://docs.pydantic.dev>
- [10] Google Gemini API Documentation – <https://ai.google.dev/gemini-api/docs>
- [11] OpenRouter API Documentation – <https://openrouter.ai/docs>
- [12] OMDb API – <https://www.omdbapi.com>
- [13] IMDb Non-Commercial Datasets – <https://developer.imdb.com/non-commercial-datasets/>